LIDR

Prompts del Módulo 4, para ti

Todo lo que se teclea en el directo, en orden, para que puedas seguirlo o rehacerlo después con calma.

> **Esto no es un guion para pegar a ciegas.** Cada prompt lleva una línea de para qué sirve, y esa línea es lo que hay que entender. En esta sesión más que en ninguna: el trabajo de hoy es **decidir qué hallazgo es real**, y eso no se puede copiar.

> ⚠️ **En el directo se dejan dos bugs sin arreglar, y aquí no.** Para que la sesión quepa en el tiempo, el mentor cierra solo el primero y deja los otros dos como deberes trazados. **Tú hazlos los tres**, cada uno con su test. Al final de este archivo tienes cómo.

> 🧭 **El orden de este archivo es el de la sesión, y el orden es contenido.** Primero se **verifica** (¿cumple lo que estaba escrito?), luego se **documenta**, y al final se **refuta** (¿qué falla aunque cumpla?). No te saltes al revisor adversarial: la mitad de la lección está en llegar a él habiendo hecho lo otro.

Antes de nada: haz FORK del repo, no lo clones

https://github.com/LIDR-academy/flowsync-ai4devs

Fork desde la web de GitHub, y después clona **tu copia**:

```
git clone git@github.com:<tu-usuario>/flowsync-ai4devs.git
cd flowsync-ai4devs
git checkout s4/start
make setup
```

⚠️ **El fork no es opcional.** Al final de la sesión le pides a tu harness que cierre el trabajo, y eso incluye **abrir un PR**. Sobre un clon directo del repo de la academia no tienes permiso de push, así que ese paso falla. Con tu fork, el PR va a tu propia copia y puedes revisarlo con calma.

La rama `s4/start` trae el harness del Módulo 1, el PRD y el backlog, y la gestión de tareas ya implementada con su spec viva en `openspec/specs/tasks/spec.md`. **Y trae tres bugs metidos a propósito.** No son un descuido: son el material de la sesión.

> 💻 **Todo se teclea desde la raíz del repo.** Los comandos de terminal van entre paréntesis — `(cd backend && …)` — para que funcionen estés donde estés y te devuelvan a la raíz. Este proyecto no tiene `package.json` en la raíz: `npm` hay que ejecutarlo dentro de `backend/` o de `frontend/`, y eso lo dice su propio `CLAUDE.md`.

Ver la suite "en verde"

En la terminal, desde la raíz del repo:

```
(cd backend && npm test)
```

Salen **20 tests en verde en un segundo**. Ahora mira **cuáles**: `signup`, `login`, `session`, `initials`. Los veinte son del login, del Módulo 1. La gestión de tareas que implementaste en el módulo anterior, con su spec y sus más de cien criterios de aceptación, **no tiene ni uno**.

Ese es el punto ciego: el semáforo no te está mintiendo, te está contando la verdad sobre **lo que ejecutó**. Lo que no puede contarte es lo que nadie le pidió que ejecutara. Y lo que tu equipo mira es el semáforo.

Sacar el inventario de lo que no está probado

```
Lee los scenarios de `openspec/specs/tasks/spec.md` y los tests que haya en
`backend/tests/`. Genera una tabla de trazabilidad: cada scenario → qué prueba lo cubre
→ cubierto / NO cubierto. Cuenta cuántos scenarios hay en total y cuántos están
cubiertos, y ponlo arriba. No cambies ningún archivo: esto es una lectura, no un
arreglo.
```

Esto es verificar: cruzar lo que el sistema **debía** hacer contra lo que alguien **comprobó** que hace. Fíjate en que la spec que escribiste en la sesión anterior te acaba de dar gratis el inventario completo de la deuda de pruebas.

Convertir un requisito en tests, y ver el rojo

```
Escribe los tests de integración que faltan para el requisito «Lo que cada tarea muestra
de su responsable» de `openspec/specs/tasks/spec.md`: uno por cada scenario, en
`backend/tests/functional/tasks/`. Sigue el estilo de los que ya existen en
`backend/tests/functional/auth/`. No toques nada fuera de `backend/tests/`: si algún
test falla, déjalo fallando, que hoy queremos saber qué está mal, no taparlo.
```

Las tres restricciones hacen falta. **Acotar a un requisito** es lo que evita que se ponga a generar cien tests que nadie va a leer. **Seguir el estilo de los de** `auth/` no es cosmético: esos aíslan con una transacción global en vez de vaciar tablas, y la suite pega contra el mismo SQLite que tu servidor de desarrollo — un test que trunque te borra tus datos. Y **"déjalo fallando"** es lo que impide que arregle el código de paso y te quedes sin ver el problema.

Después, `(cd backend && npm test)`. Sale **rojo**, y lo ha encontrado la spec: nadie ha revisado nada todavía.

⚠️ **Para aquí y mira dónde ha dejado el trabajo**, con `git branch` y `git status --short`. Vas a ver una de dos cosas, y depende del modelo que uses:

Lo que ves	Qué ha pasado
Una rama nueva, con el trabajo commiteado ahí	Cumplió tu regla **al pie de la letra, por petición**. No es un fallo suyo: lo manda tu `AGENTS.md` del Módulo 1
Sigues en `s4/start` y `git status` lleno	**Se saltó la regla entera**, y tu trabajo no está guardado en ninguna parte: un `git checkout` se lo lleva

Sea cual sea, no sigas sin leer el apartado siguiente. El primer caso acaba con el día repartido en seis ramas; el segundo, con el día sin guardar.

Afinar la regla antes de que se te llene de ramas

Tu regla dice "*antes de tocar código, crear una rama nueva*", y no dice qué es "*una tarea*". Si el agente la cumple, la cumple **al pie de la letra**: una rama por cada cosa que le pidas. Está medido — con la cadena entera y la regla sin tocar, el trabajo acaba en **seis ramas paralelas y ninguna lo tiene entero**.

Y si te ha tocado el otro caso —ni rama ni commits— fíjate en lo que acabas de ver, porque es la sesión entera en un ejemplo: **escribiste una regla, está documentada, y no se ha cumplido.** Una cosa es que algo esté escrito y otra que ocurra. Por eso se verifica.

No lo arregles en el prompt: lo tendrías que repetir en los diez que quedan. Arréglalo donde el agente lee siempre.

```
Afina las reglas de proceso de `CLAUDE.md` (`AGENTS.md` es un symlink que apunta ahí):
la rama es por unidad de trabajo, no por petición — si ya estás en una rama de trabajo
que no sea `main` ni una `sN/*`, sigue en ella en vez de crear otra. El commit sí es por
petición, con la skill `/commit`. Y `gh pr create` y el pase del subagente
`adversarial-reviewer` van una sola vez, al terminar la unidad de trabajo, no al cerrar
cada petición. No cambies nada más del archivo.
```

Fíjate en lo que la regla **conserva**: el commit sigue siendo por petición. Commits atómicos es lo que hace legible un `git log`; lo que sobraba era la rama. Afinar una regla no es tirarla.

Y de paso: `AGENTS.md` es un **symlink** a `CLAUDE.md` (`ls -la AGENTS.md`). Un archivo, dos nombres, porque cada herramienta busca el suyo.

Antes de arreglar nada, abre `backend/app/transformers/task_transformer.ts` y lee el comentario que hay encima. El bug se introdujo **quitando duplicación**, que parece buena práctica, y se llevó por delante un requisito de seguridad escrito dos líneas más arriba. Así es como se filtra un dato personal en producción: no con una chapuza, con un refactor razonable.

Arreglar lo que el rojo destapó

```
Arregla el fallo que acaban de destapar los tests del responsable, y no toques nada más. El comportamiento correcto está en ese mismo requisito de `openspec/specs/tasks/spec.md`: la tarea lleva el nombre y las iniciales de su responsable, y no lleva su email. No cambies los tests para que pasen: el que tiene que cambiar es el código.
```

La última frase cierra el atajo clásico: aflojar la aserción hasta que el rojo desaparezca. Es lo que haría cualquiera con prisa, y es exactamente lo que convierte una suite en decoración.

`(cd backend && npm test)` otra vez: verde, y ahora con los tres tests nuevos dentro.

Generar el diagrama de arquitectura

```
Analiza la estructura real de este proyecto (rutas, controladores, modelos, transformers, la capa de datos y el frontend de React) y genera un diagrama de contenedores estilo C4 en sintaxis Mermaid. Guárdalo en `docs/architecture.md`, con un párrafo corto encima que explique qué muestra. Usa solo lo que exista en el código: si algo no lo puedes verificar leyendo archivos, no lo dibujes.
```

Sin la última frase te dibuja cachés y colas que este proyecto no tiene, porque completa con la arquitectura que **suele** tener un proyecto así. Ábrelo en GitHub: se renderiza como diagrama, vive en el repo y se ve en el diff.

Generar el contrato de la API desde el código

En la terminal, desde la raíz del repo:

```
(cd backend && npm install @foadonis/openapi && node ace configure @foadonis/openapi)
```

Y después, porque las rutas **no** se registran solas:

```
Registra las rutas del documento OpenAPI en `start/routes.ts` usando
`openapi.registerRoutes()`, tal como pide la documentación del paquete que acabamos de
configurar, y cambia en `config/openapi.ts` el título y la versión que dejó el configure
por defecto para que nombren a FlowSync. Después dime en qué URL exacta queda servido el
documento y en qué URL la interfaz, leyéndolo de la configuración y no de lo que suele
ser por defecto.
```

El documento queda en `/api.json` (y `/api.yaml`), y la interfaz en `/api`. No en `/openapi`, que es donde casi todo el mundo lo busca primero.

Si quieres cerrar el círculo:

```
Contrasta el documento OpenAPI servido en `/api.json` con los scenarios de
`openspec/specs/tasks/spec.md`. Dime dos cosas por separado: primero, qué exige la spec
que el documento NO recoge (parámetros, códigos de respuesta, forma de los objetos); y
segundo, si algún scenario de la spec no tiene ningún endpoint que lo sirva. Cita el
scenario en cada caso.
```

Cerrar la lista: que el documento recoja de verdad la spec

El paso anterior te da la lista de lo que falta. Este la cierra.

```
Cierra ahora la lista que acabas de darme: decora los controladores de tasks con los
decoradores de `@foadonis/openapi` para que el documento recoja lo que la spec exige —
el parámetro `status` con sus valores, los códigos de respuesta de cada operación y la
forma de los objetos que devuelve. Los esquemas que se repitan van en
`backend/app/openapi/schemas.ts` y se publican en `components.schemas`, para que el
documento los describa una vez y cada respuesta los referencie. No cambies rutas,
validadores ni comportamiento: hoy solo se anota lo que el código ya hace. Al terminar,
vuelve a servir el documento y dime qué puntos de la lista quedan cubiertos y cuáles no.
```

Sin esto el documento se queda casi vacío — `responses: {}`, `parameters: []`, `components.schemas: {}` — y entonces no es un contrato, es un índice de rutas. Fíjate en las dos restricciones: **la ruta de los esquemas va escrita** porque hay pasos que necesitan saber dónde vive ese archivo, y **"no cambies rutas ni validadores"** porque anotar y refactorizar se parecen demasiado cuando hay prisa. Anotar describe lo que el código ya hace; no lo cambia.

Escribir el ADR de una decisión que ya tomaste

```
Escribe un ADR en formato Nygard, con sus cinco apartados (título, contexto, decisión, estado, consecuencias), que registre la decisión de usar las delta-specs de OpenSpec como fuente de verdad viva de este proyecto. Usa como contexto lo que hay de verdad en `openspec/`: las specs, los changes archivados y cómo se relacionan. Guárdalo en `docs/adr/0001-openspec-como-fuente-de-verdad.md`. En consecuencias no pongas solo ventajas: incluye lo que esta decisión nos cuesta.
```

Fíjate en la dirección: el agente **no decide**, redacta una decisión que ya estaba tomada, con contexto que ya estaba en el repo. Si le preguntas "*¿qué deberíamos usar como fuente de verdad?*" te dará una respuesta convincente sobre algo que nadie ha decidido, y eso no es un ADR: es una opinión con formato.

Y el relevo, que es lo que casi nadie hace bien

```
Supongamos que dentro de un año dejamos de mantener las specs de OpenSpec y pasamos a que los tests de integración sean la única fuente de verdad ejecutable. Escribe el ADR que registra ese cambio en `docs/adr/0002-tests-como-fuente-de-verdad-ejecutable.md`, y actualiza el estado del ADR 0001 para que quede marcado como reemplazado por este, con el enlace. No borres ni reescribas el contenido del 0001: su contexto y su decisión siguen siendo lo que se creía entonces.
```

Un ADR **no se edita** cuando la decisión cambia: se marca como reemplazado y se escribe otro. Lo que se conserva es lo valioso, porque el 0001 sigue explicando por qué aquello tenía sentido con lo que se sabía entonces. Eso es lo que te ahorra repetir un debate que ya tuviste.

Y el ancla tiene que sobrevivir a que alguien borre el código: el PR y el commit sirven, y los changes archivados de OpenSpec también. **Un comentario en el código no sirve**, porque se va con el archivo.

El README que no envejece

```
Genera el README de la capability de tareas en `docs/capabilities/tasks/README.md`: qué hace, qué endpoints expone, qué reglas de negocio tiene y cómo se prueba en local. Sácalo del código y de `openspec/specs/tasks/spec.md`, y enlaza la spec en vez de copiar sus reglas, para que no haya dos sitios donde se puedan desincronizar.
```

"*Enlaza en vez de copiar*" es toda la diferencia entre un README que miente en tres meses y uno que no. Y **la ruta va escrita a propósito**: si dejas que el agente la elija, elige una razonable distinta cada vez, y el paso siguiente necesita saber dónde vive el archivo.

Que no dependa de que te acuerdes

Aquí es donde la mayoría se queda a medias: genera la documentación una vez, y en tres meses vuelve a estar mintiendo. Lo que lo evita no es tu disciplina, es tu harness.

```
Añade a `CLAUDE.md`, en la sección de reglas de proceso, una regla corta: cuando un cambio toque rutas, controladores, validadores o transformers de una capability, en el mismo commit hay que regenerar el documento OpenAPI y actualizar el README de esa capability en `docs/capabilities/<nombre>/README.md`. Escríbela con el mismo tono y formato que las reglas que ya están ahí, y no añadas ninguna otra.
```

Fíjate dónde acaba esa regla: en el mismo archivo donde escribiste en el Módulo 1 "*antes de tocar código, crea una rama*". Esa regla lleva cuatro módulos cumpliéndose sin que la recuerdes — hoy la has visto actuar cuando el agente ramificó solo. La documentación viva se mantiene igual: escrita una vez, en el sitio donde el agente lee.

El escalón siguiente es el mismo patrón un paso más allá: esto mismo como job de CI, igual que el hook de formateo de la primera sesión. Guardarraíles, no buena voluntad.

Cerrar

Mira lo que has construido, en una sola rama:

```
git log --oneline s4/start..HEAD
git status
```

Cada petición dejó su commit y el árbol está limpio. Eso es docs-as-code: la documentación vive en el repo, se commitea con el código y se revisa en el mismo PR.

```
Ya está todo el trabajo de hoy en esta rama. Abre el PR con `gh pr create` y escribe una descripción que cuente la historia completa: qué se verificó, qué se arregló, qué documentación se generó y qué regla nueva lleva el harness. No abras más de uno.
```

Un minuto: tu documentación ya está publicada

Abre el PR que acabas de crear, o la carpeta `docs/` de tu repo en GitHub, y mira dos cosas:

- **`docs/architecture.md`**: el bloque de Mermaid **se dibuja solo**. GitHub lo renderiza sin que instales nada.

- `docs/capabilities/tasks/README.md`: los enlaces a la spec y al código **funcionan**. Se navega de la documentación al código y vuelta.

Eso ya es un sitio de documentación, y no has instalado ningún generador. Y no puede quedarse atrás, porque nace del mismo commit que el código.

Sin vender de más: esto es **el repo renderizado**, no un portal publicado — quien no tenga acceso al repo no lo ve. El escalón siguiente es GitHub Pages con un job de CI, y ahí hay un detalle que sorprende: **publicar el Markdown tal cual no basta**, llega al navegador como texto plano. Hace falta un generador de sitios (MkDocs, Docusaurus). Está medido, no supuesto.

Afilar tu revisor adversarial

> 📌 **Esto va al final por una razón, y no es que sobre tiempo.** Abre tu `AGENTS.md` del Módulo 1, sección de reglas de proceso: dice que después de abrir el PR hay que pasarle el subagente adversarial antes de dar el trabajo por terminado. Acabas de abrir el PR. Lo que viene ahora **lo mandaste tú hace cuatro módulos**.
>
> Y hay una segunda razón, de fondo: lo que has hecho hasta aquí es **verificar** — comprobar que el código cumple lo que estaba escrito. Lo que viene busca **lo que nadie llegó a escribir**. No son el mismo trabajo, y por eso van separados.

Ya tienes uno en `.claude/agents/adversarial-reviewer.md` desde el Módulo 1, pero está pensado para revisar el login. Déjalo así:

```
---
name: adversarial-reviewer
description: Revisa un cambio con el objetivo de REFUTARLO. Contrasta contra la spec
viva de OpenSpec. Read-only.
tools: Read, Grep, Glob
model: sonnet
---
Eres un revisor adversarial. Tu único objetivo es DEMOSTRAR que el código está mal,
no aprobarlo. Contrasta cada cambio contra los scenarios de `openspec/specs/`.
Busca: (1) desviaciones de la spec, (2) edge cases no manejados, (3) fugas de
seguridad o de datos, (4) supuestos frágiles. Devuelve hallazgos PRIORIZADOS
(crítico/alto/medio) con evidencia (archivo:línea) y el scenario que se rompe.
No edites archivos. Si no encuentras nada, busca más a fondo.
```

Dos cosas cambian, y las dos importan. **"Contrasta contra los scenarios de la spec"**: sin eso revisa contra su idea de lo que es buen código, que es una opinión de internet; con eso revisa contra tu contrato. **"Si no encuentras nada, busca más a fondo"**: le quita la salida fácil de aprobar.

Y lo que **no** cambia: sigue siendo `Read, Grep, Glob`. De solo lectura, para que no pueda arreglar un bug y esconderlo.

Soltarlo

```
Usa el subagente `adversarial-reviewer` contra la capability de tareas: el modelo `Task`, sus transformers, sus validators y los controladores que sirven `GET /api/v1/tasks` y `GET /api/v1/tasks/:id`. No arregles nada: hoy solo queremos el informe.
```

Fíjate en lo corto que es. Contrastar contra la spec, citar el scenario roto, dar severidad y `archivo:línea`, no editar nada: todo eso ya está en el archivo del subagente que acabas de escribir. Repetirlo en el prompt no lo refuerza, solo te acostumbra a no fiarte de tu propio harness. Lo que queda aquí es lo que cambia cada vez: **qué** hay que mirar.

Tarda. Es el paso más lento de la sesión, y no se arregla poniendo un modelo más rápido.

⚠️ **No esperes un número concreto de hallazgos.** Cambia entre ejecuciones. Lo que no cambia es la forma: severidad, archivo y línea, y el scenario que se rompe.

Lo importante: decidir cuál es real

Cuando tengas los hallazgos, **no los apliques**. Léelos con la spec abierta al lado, y compáralos con el que arreglaste esta mañana:

- El **email** del responsable en cada tarea de la lista — el que destaparon los tests, y que ya está corregido.
- Que la lista devuelve **tareas de otras personas** sin filtrar por quién la pide.

Los dos suenan igual de mal. **Solo uno era un bug.** Busca en `openspec/specs/tasks/spec.md` el requisito "*Una sola lista compartida del espacio*" y su scenario "*Las tareas ajenas también salen*", y luego el requisito sobre lo que cada tarea muestra de su responsable.

La lista compartida **es el producto**: FlowSync existe para que un equipo vea en qué anda cada uno. Si "arreglas" eso, rompes la feature principal y encima pasas el code review, porque el cambio parece que mejora la seguridad. El email, en cambio, está prohibido por escrito.

⚠️ **Puede que el revisor ni siquiera lo mencione.** En los ensayos no lo marcó, y eso también es información: no marcarlo fue correcto. La pregunta te la tienes que hacer tú, y ese es exactamente el músculo del día.

Quédate con esto, que es lo que se lleva de hoy: **un bug y una decisión de producto se parecen muchísimo en el código.** Lo que los distingue no es el criterio del agente. Es que exista un documento

que diga qué se espera del sistema.

Tus deberes: los dos hallazgos que el directo no arregla

Los dos están trazados y sabes qué scenario rompen. Hazlos igual que el crítico: el arreglo **y** el test que lo blinda.

1. **`isOverdue` sale `true` aunque la tarea esté hecha.** Está en `backend/app/models/task.ts`. La regla la tienes en la spec: una tarea hecha no está vencida, aunque su fecha haya pasado. Es el mismo edge case que estudiaste en el asíncrono.
2. **El filtro por estado no valida el valor.** Está en `backend/app/validators/task.ts`. Pide `GET /api/v1/tasks?status=inventado` y mira lo que te contesta: un `200` con una lista vacía, que es exactamente el fallo silencioso que la spec quería impedir. Debería ser un `422`.

El segundo es el más instructivo de los dos: no revienta, no aparece en ningún log, y un cliente que lo llame mal se lleva "no hay tareas" y se lo cree.